ChatGPT

I am is looking at implementing a redis opensource cluster deployment. Can you come up with Docker files, helm charts etc which I can deploy in docker kind and validate the implementation. I cannot use redis enterprise as it is very expensive for me. Basically what I am looking for is consistent cache across all redis pods or nodes, cache persistence etc.

Thought for 3m 0s

I put together a starter package for a **Redis Open Source Cluster on kind** with:

- Helm chart
- kind cluster config
- Docker Compose fallback
- bootstrap/init/validation scripts
- persistence enabled with **AOF + RDB**
- validation job for cluster health and key read/write

Files:
Download the project folder as tar.gz

Key point before you deploy: with **Redis OSS Cluster**, you do **not** get the exact same full cache on every pod. Redis Cluster shards keys across masters and uses replicas for redundancy/failover, so consistency is per key/slot ownership rather than “every node holds all keys.” Redis Cluster uses 16384 hash slots, replicas mirror their assigned master, and Redis persistence supports AOF and RDB, with AOF used for recovery when both are enabled. Redis +2

I also built the package around kind-friendly persistence. For local testing, `hostPath`-style persistence is acceptable, while Kubernetes generally recommends PersistentVolumes / StorageClasses for broader use. Kubernetes +1

One more Redis Cluster detail this chart handles: cluster nodes need both the normal Redis port and the cluster bus port, which by default is the data port plus 10000. Redis

A few honest caveats:

- I could generate the full package here, but I could not run a live `helm install` / `kind create cluster` validation in this environment because `helm`, `kind`, and `kubectl` are not installed in the container.
- So treat this as a solid starter implementation, but expect one or two environment-specific adjustments during your first local run.

The README in the package has the exact install and validation flow. The fastest path is:

1. create the kind cluster with `kind-config.yaml`
2. install the Helm chart
3. wait for the StatefulSet and init job
4. run the built-in validation job
5. test persistence by setting a key, deleting a pod, and reading the key back

For your requirement, this is the most practical open-source approach without Redis Enterprise:

- **scalable cache** via Redis OSS Cluster
- **durability** via AOF + RDB
- **failover** via replicas
- **local validation** in kind

If your actual requirement is "every Redis node must contain the same full dataset," then you want a **single primary + replicas / Sentinel-style** topology instead of Redis Cluster.

I can also produce a ↪ **production-oriented version** next with auth, TLS, anti-affinity, PDB, Prometheus exporter, and dynamic PVCs for EKS/OpenShift.

Sources

Okey I have openshift cluster for deployment. Can you implement the same for openshift.

Thought for 3m 4s ›

Yes — I created an **OpenShift-ready Redis OSS Cluster package** for you.

Download:
redis-oss-openshift.tar.gz

What's inside:

- Helm chart for Redis OSS Cluster on OpenShift
- OpenShift-friendly security settings
- StatefulSet + headless service
- PVC-based persistence
- bootstrap Job to form the cluster
- validation Job
- helper scripts for install and validation
- example values override file

A few important points:

Redis OSS Cluster will **not keep identical full cache contents on every pod**. It shards keys across masters and keeps replicas for redundancy/failover, so "consistency" is by slot ownership plus replication, not "same full dataset on all pods." Redis Cluster is designed for sharding, asynchronous replication, failover, and client redirection via `MOVED` / `ASK`. Redis +1

For persistence, this package enables **AOF + RDB**, which Redis documents as supported together. RDB gives point-in-time snapshots and AOF replays writes at startup. Redis +1

For OpenShift compatibility, I designed it around:

- **StatefulSet** for stable network/storage identity. Red Hat Docum... +1
- A **headless Service** with `publishNotReadyAddresses: true` so pod DNS names resolve during bootstrap. Red Hat Docum...
- Avoiding fixed `runAsUser`, while using `allowPrivilegeEscalation: false`, dropped capabilities, and seccomp defaults that align with OpenShift's default `restricted-v2` posture. Red Hat Docum...

Also, Redis Cluster requires both the normal Redis port and the **cluster bus port** (normally data port + 10000), which the chart exposes internally. Redis +1

Suggested install flow:

Bash

```bash
tar -xzf redis-oss-openshift.tar.gz
cd redis-oss-openshift
./scripts/install-openshift.sh redis-oss redis-oss
./scripts/wait-and-validate.sh redis-oss redis-oss
```

Honest caveat: I could build the package here, but I could not run a live OpenShift deployment from this environment, so you may need small adjustments for your cluster's StorageClass, image policy, or SCC nuances.

If your actual requirement is **the same dataset on every node**, I should build you an **OpenShift Redis primary-replica + Sentinel** package instead, because Redis Cluster is not the right topology for that.

Sources

when a client reaches the redis will it have same data regardless of the behind the scenes implementation. Let us say client fetches xyz key from redis will which pod it will be routed?

Thought for 5s

Not automatically, no.

With **Redis OSS Cluster**, the keyspace is **sharded** across masters by hash slot. A given key like `xyz` belongs to exactly one slot, and that slot is owned by one master at a time. Replicas hold copies of that master's data for failover, but the whole dataset is **not present on every pod**. Redis +1

So for your example:

- Client asks for key `xyz`
- Redis computes the hash slot for `xyz`
- That slot maps to one specific master node
- The request ultimately goes to **that master's pod** (or sometimes a replica if you explicitly use replica reads in your client/setup) Redis +2

In practice, which pod gets hit depends on the client:

1. **Cluster-aware client**
   The client knows the slot map and sends `GET xyz` directly to the pod that owns `xyz`'s slot. It typically learns this mapping via cluster metadata such as `CLUSTER SLOTS`. Redis +1

2. **Client connects to the "wrong" pod first**
   That pod replies with a `MOVED` redirection telling the client which node actually owns the slot, and the client retries there. During migration/resharding you can also see `ASK`. Redis +1

So the answer to "which pod will it be routed to?" is:

- **the pod owning the hash slot for** `xyz`, usually a master pod
- not a random pod
- not all pods
- not necessarily the pod behind the service IP the client first contacted Redis +1

About "will it have the same data regardless of behind-the-scenes implementation?":

- For **the same key**, the client should get the same current value as long as it reaches the correct slot owner or an up-to-date replica. Redis +1
- But Redis Cluster replication is not fully strong consistency. Replication is asynchronous by default, so right after a write there can be a short window where a replica is behind, and in rare failure cases writes can be lost during failover. Redis documents that Redis Cluster does **not** provide strong consistency even with `WAIT`. Redis +1

So the practical takeaway is:

- **Normal case:** `GET xyz` returns the same value regardless of which node the client initially connects to, because the client will be routed to the correct slot owner. Redis +1
- **But:** the value is not served from every pod having a full identical cache. It is served from the shard that owns that key, with replicas for HA. Redis +1

If your real requirement is:

> "Any Redis pod should be able to answer any key locally because all pods have the same full dataset"

then **Redis Cluster is not the right topology**. You would want **single primary + replicas + Sentinel** for HA, where replicas are intended to be copies of the primary's dataset. Redis +1

A clean way to think about it:

- **Redis Cluster** = scale out, sharded data
- **Primary + replicas + Sentinel** = same dataset replicated, HA without sharding Redis +1

I can next give you ↳ a simple architecture recommendation for your use case: **Cluster vs Sentinel** based on whether you care more about scale or same-data-on-all-nodes.

Sources

Give me a simple architecture recommendation for my use case: Cluster vs Sentinel based on whether I care more about scale or same-data-on-all-nodes

Thought for 4s

For **your use case**, I'd recommend this:

Recommendation

Choose Redis Sentinel if your priority is:

- **same dataset available on all Redis nodes**
- **simple client behavior**
- **HA + failover**
- **cache persistence**
- moderate scale that fits on **one primary dataset**

Why:
With Sentinel, Redis runs as **primary + replicas**. Replicas keep copies of the primary's dataset, and Sentinel monitors them and promotes a replica if the primary fails. Redis documents Sentinel as the HA solution for **non-clustered Redis**, while replication keeps replicas in sync with the primary. Redis +2

This matches your statement best:

> "consistent cache across all redis pods or nodes"

That is much closer to **primary/replica** than to Redis Cluster.

Choose Redis Cluster if your priority is:

- **horizontal scale**

- dataset larger than one Redis node can handle
- higher write/read throughput by **sharding**
- app/client can use a **cluster-aware Redis client**

Why:
Redis Cluster automatically shards data across nodes and gives HA through replicas per shard, but the full dataset is **not present on every node**. A key is stored on the shard that owns its hash slot. Redis explicitly positions Cluster for scaling via sharding. Redis +2

My simple decision rule

Pick Sentinel when:

- your app mostly wants Redis as a **shared cache/service endpoint**
- you want **any failover target to have the same logical dataset**
- you do **not** need sharding right now
- operational simplicity matters more than max scale

Pick Cluster when:

- one Redis node is not enough for **memory or throughput**
- you expect to grow cache size significantly
- you are okay with **data being split across shards**
- your client libraries fully support Redis Cluster

What I would suggest for you

Based on what you asked so far, I would go with:

Redis OSS + Sentinel + primary/replicas + AOF/RDB persistence

Because it gives you:

- same logical cache dataset on replicas
- automatic failover
- simpler routing model for clients
- persistence support
- easier OpenShift deployment and testing than full cluster sharding

---

One practical note

Even with Sentinel, writes go to the **primary** and replicas catch up through replication. So this is not “multi-writer same-data-on-all-nodes.” It is:

- one writable primary
- one or more replicas
- automatic promotion on failure

Replication is asynchronous by default, so there can still be a brief lag window. Redis notes this explicitly. Redis +1

Final answer

For your requirement, **Sentinel is the better fit**.
Use **Cluster only if scale is the main driver** and you accept sharded data.

If you want, I’ll build you the **OpenShift deployment package for Redis Sentinel** next — Helm chart, manifests, PVCs, failover test, and kind-compatible local validation.

Sources

can you give me redis sentinel implementation

Thought for 2m 9s

Yes — I created an **OpenShift Redis Sentinel implementation package** for you.

Download:
redis-sentinel-openshift.tar.gz

What it includes:

- Helm chart for **Redis OSS primary + replicas + Sentinel**
- OpenShift-friendly security settings
- StatefulSet for Redis
- Deployment for Sentinels
- PVC-backed persistence

- validation job
- failover test script
- example values for OpenShift

Why this fits your use case:

- Redis replication makes replicas **exact copies of the primary's dataset** in normal operation. Redis
- Redis Sentinel provides **automatic failover** and acts as a **configuration provider/service discovery** layer for clients. Redis
- This is a better match than Redis Cluster when you care more about **same logical data across nodes** than horizontal sharding. Redis Cluster is specifically for sharded scale-out. Redis +1
- OpenShift uses **Security Context Constraints**, so the chart avoids privileged settings and fixed UIDs to stay friendly to restricted environments. Red Hat Docum...

Important behavior to keep in mind:

- Your app should typically **write to the current primary**.
- Replicas are there for **HA and failover**, not active multi-writer use.
- Replication is **asynchronous by default**, so there can be a brief lag window after writes. Redis

Suggested flow:

Bash

```bash
tar -xzf redis-sentinel-openshift.tar.gz
cd redis-sentinel-openshift
./scripts/install-openshift.sh redis-sentinel redis-sentinel
./scripts/wait-and-validate.sh redis-sentinel redis-sentinel
./scripts/failover-test.sh redis-sentinel redis-sentinel
```

One honest caveat: I generated the implementation package here, but I could not run a live OpenShift deployment in this environment, so you may need small adjustments for your cluster's storage class, SCC policy, or image pull policy.

The package is intentionally simple and practical. The next useful step would be a **production-hardened version** with:

- Redis auth
- TLS
- NetworkPolicies
- PodDisruptionBudget

- anti-affinity
- metrics/exporter
- more robust primary service relabeling after failover

Sources

what exactly is sentinel

Let's break this down simply — **Redis Sentinel is not Redis itself**, it's a **monitoring + failover + discovery system** that sits alongside Redis.

🧠 What exactly is Redis Sentinel?

Redis Sentinel = "watchdog + auto-recovery system" for Redis

It manages a **primary + replicas setup** and ensures your system keeps working even if the primary goes down.

🔧 What Sentinel actually does

1. 🩺 Monitoring

Sentinel constantly checks:

- Is Redis primary alive?
- Are replicas healthy?

It uses heartbeats (PINGs) to detect failures.

---

2. 🔄 Automatic Failover

If primary goes down:

1. Sentinel detects failure
2. Multiple Sentinels agree (quorum)
3. One replica is promoted to **new primary**

4. Other replicas reattach to new primary

👉 This happens automatically — no manual intervention

---

3. 📡 Service Discovery

Your application doesn't need to know which pod is primary.

Instead, it asks Sentinel:

> "Who is the current master?"

Sentinel responds with:

```
IP + PORT of current primary
```

---

4. 🚨 Notification (optional)

Sentinel can:

- send alerts
- trigger scripts/webhooks

🏗️ Architecture (simple view)

```
+------------------+
|   Sentinel (1)   |
+------------------+
         |
+------------------+
|   Sentinel (2)   |
+------------------+
         |
+------------------+
|   Sentinel (3)   |
+------------------+

         ↓ monitors

+------------------+
```

```
        |   Redis Primary   |
        +-------------------+
            ↑         ↑
            |         |
    +--------------+  +--------------+
    |  Replica 1   |  |  Replica 2   |
    +--------------+  +--------------+
```

📦 What Sentinel is NOT

❌ Not a data store
❌ Not a cache itself
❌ Not a load balancer

It only **manages Redis nodes**

🔁 How your app interacts

Without Sentinel (bad for HA)

```
App → Redis primary (hardcoded)
```

If primary dies → ❌ app breaks

With Sentinel (recommended)

```
App → Sentinel → (gets current primary) → Redis
```

If primary changes → ✅ app continues working

⚠️ Important behavior

Writes

- Always go to **primary**

Reads

- Can go to:
  - primary (default)
  - replicas (if you configure it)

Consistency

- Replication is **asynchronous**
- Small delay possible between primary and replicas

🔑 Why Sentinel fits your use case

You wanted:

> "consistent cache across all nodes"

With Sentinel:

- All replicas contain **same dataset (eventually consistent)**
- Failover is automatic
- No sharding complexity
- Easier client logic

🆚 Sentinel vs Cluster (quick clarity)

Feature	Sentinel	Cluster
Data	Same dataset	Sharded
Scale	Vertical	Horizontal
Failover	Yes	Yes
Complexity	Low	High
Best for	HA cache	Large distributed cache

🧩 Real-world analogy

- **Redis = database server**
- **Sentinel = Kubernetes for Redis nodes**

It:

- monitors
- restarts (via failover)
- redirects traffic

🧠 Final one-line definition

👉 **Redis Sentinel is a distributed system that monitors Redis instances and automatically handles failover while telling clients where the current primary is.**

If you want next step, I can:

- ↳ show **exact request flow (client → sentinel → redis)**
- or ↳ help you integrate Sentinel with **your app (Java / Python / Node)**
- or ↳ explain **how failover actually happens internally step-by-step**

My organization only provided rpm for redis 7.2, rhel8 with open jdk 21. Can you also add a docker file? Also why have you not provided redis.conf anywhere?

Yes — and you're right to call that out.

I should have included explicit `redis.conf` and `sentinel.conf` instead of relying too much on command-line flags and generated runtime config. For a real deployment, keeping those as first-class files is better because it makes persistence, replication, failover, and OpenShift/container behavior much easier to reason about.

For your environment:

- **Redis 7.2 RPM**
- **RHEL 8 base**
- **OpenJDK 21 available in org image stack**

you can absolutely build a Redis Sentinel image from your internal RPM.

Why `redis.conf` was missing before

There are usually three ways people configure Redis in containers:

1. pass flags in the container command
2. generate config at startup from env vars
3. mount explicit config files

I leaned toward the first two for portability, but for your case that was not ideal.

For **Sentinel-based HA**, you really want:

- `redis.conf` for Redis server behavior
- `sentinel.conf` for Sentinel monitoring behavior
- an entrypoint that patches a few runtime values like pod IP, hostname, replica target, announce settings

That makes troubleshooting much easier.

Recommended container structure

Use one image that contains:

- Redis binaries from your org RPM
- optional Java 21 only if your org base image requires it
- `redis.conf`
- `sentinel.conf`
- `entrypoint.sh`

Then run the same image in either:

- **redis mode**
- **sentinel mode**

Example Dockerfile

Assuming you have an internal RPM available in the build context as `redis-7.2*.rpm`.

dockerfile

```dockerfile
FROM registry.access.redhat.com/ubi8/ubi:8.10

USER root

RUN dnf -y update && \
    dnf -y install hostname shadow-utils procps-ng iproute findutils tar gzip &&
    dnf clean all

# Optional: only keep this if your org standard image requires Java 21 in the sam
RUN dnf -y install java-21-openjdk-headless && \
    dnf clean all

COPY redis-7.2*.rpm /tmp/redis.rpm

RUN dnf -y install /tmp/redis.rpm && \
    rm -f /tmp/redis.rpm && \
    dnf clean all

RUN mkdir -p /opt/redis/conf /opt/redis/scripts /data && \
    chmod -R g=u /opt/redis /data

COPY conf/redis.conf /opt/redis/conf/redis.conf
COPY conf/sentinel.conf /opt/redis/conf/sentinel.conf
COPY scripts/entrypoint.sh /opt/redis/scripts/entrypoint.sh

RUN chmod +x /opt/redis/scripts/entrypoint.sh

EXPOSE 6379 26379

USER 1001

ENTRYPOINT ["/opt/redis/scripts/entrypoint.sh"]
CMD ["redis"]
```

Example `redis.conf`

This is a good baseline for Sentinel topology with persistence.

conf

```
bind 0.0.0.0
protected-mode no
port 6379

dir /data
dbfilename dump.rdb

appendonly yes
appendfilename "appendonly.aof"
appendfsync everysec

save 900 1
save 300 10
save 60 10000

loglevel notice
logfile ""

tcp-keepalive 300
timeout 0

replica-read-only yes
repl-diskless-sync yes
repl-diskless-sync-delay 5

lazyfree-lazy-eviction yes
lazyfree-lazy-expire yes

# Optional memory policy
# maxmemory 1gb
# maxmemory-policy allkeys-lru
```

Why these matter

- `appendonly yes` gives durable write persistence
- `appendfsync everysec` is a practical balance
- `save ...` keeps RDB snapshots too
- `logfile ""` sends logs to stdout in containers
- `protected-mode no` is fine only inside cluster network / secured namespace

---

Example `sentinel.conf`

conf

```
port 26379
bind 0.0.0.0
protected-mode no

logfile ""
dir /tmp

sentinel resolve-hostnames yes
sentinel announce-hostnames yes

# These lines are patched dynamically by entrypoint if needed
sentinel monitor mymaster redis-0.redis-headless.redis.svc.cluster.local 6379 2
sentinel down-after-milliseconds mymaster 5000
sentinel failover-timeout mymaster 60000
sentinel parallel-syncs mymaster 1
```

Meaning

- `mymaster` is just the monitored primary name
- quorum `2` means at least 2 sentinels must agree
- `down-after-milliseconds` controls failure detection
- `parallel-syncs` limits how many replicas resync at once after failover

Example `entrypoint.sh`

This lets the same image run as either Redis or Sentinel.

Bash

```bash
#!/usr/bin/env bash
set -euo pipefail

MODE="${1:-redis}"
REDIS_CONF="/opt/redis/conf/redis.conf"
SENTINEL_CONF="/opt/redis/conf/sentinel.conf"

POD_NAME="${HOSTNAME:-redis-0}"
POD_IP="$(hostname -i | awk '{print $1}')"

MASTER_NAME="${MASTER_NAME:-mymaster}"
MASTER_HOST="${MASTER_HOST:-redis-0.redis-headless.redis.svc.cluster.local}"
MASTER_PORT="${MASTER_PORT:-6379}"
QUORUM="${SENTINEL_QUORUM:-2}"
```

```
echo "Starting mode=${MODE} pod=${POD_NAME} ip=${POD_IP}"

if [[ "${MODE}" == "redis" ]]; then
  ORDINAL="${POD_NAME##*-}"

  cp "${REDIS_CONF}" /tmp/redis.conf

  cat >> /tmp/redis.conf <<EOF
replica-announce-ip ${POD_NAME}.redis-headless.redis.svc.cluster.local
replica-announce-port 6379
EOF

  if [[ "${ORDINAL}" != "0" ]]; then
    cat >> /tmp/redis.conf <<EOF
replicaof ${MASTER_HOST} ${MASTER_PORT}
EOF
  fi

  exec redis-server /tmp/redis.conf

elif [[ "${MODE}" == "sentinel" ]]; then
  cp "${SENTINEL_CONF}" /tmp/sentinel.conf

  sed -i "s|^sentinel monitor .*|sentinel monitor ${MASTER_NAME} ${MASTER_HOST} $

  cat >> /tmp/sentinel.conf <<EOF
sentinel announce-ip ${POD_NAME}.sentinel-headless.redis.svc.cluster.local
sentinel announce-port 26379
EOF

  exec redis-sentinel /tmp/sentinel.conf

else
  echo "Unknown mode: ${MODE}"
  exit 1
fi
```

How this would work in OpenShift

You would typically run:

Redis StatefulSet

- `redis-0` starts as primary
- `redis-1`, `redis-2` start as replicas using `replicaof redis-0 ...`

Sentinel Deployment or StatefulSet

- 3 Sentinel pods
- all monitor `mymaster`
- on primary failure, they elect and promote a replica

Why explicit config files are better here

For your org setup, explicit config files help because:

- RPM installs may place binaries/config differently
- OpenShift troubleshooting is easier when configs are visible
- persistence settings should not be buried in long command args
- Sentinel behavior is much easier to audit in a real file

One important note about Docker + OpenShift

OpenShift usually runs containers with a random non-root UID. So make sure:

- config files are readable by arbitrary UID
- `/data` is writable
- group permissions are open enough

That is why in the Dockerfile I used:

```dockerfile
chmod -R g=u /opt/redis /data
USER 1001
```

This is OpenShift-friendly.

Recommended deployment model for you

Given your requirement, I would now structure it like this:

- **1 image**
- **2 config files**
- **1 entrypoint**
- deploy same image as:

- Redis server pods
- Sentinel pods

That keeps everything simple and consistent with your internal RPM approach.

My recommendation on Java 21

Unless your organization mandates a common Java base image, Redis itself does **not** need OpenJDK 21.

So better options are:

- **preferred:** Redis image based only on UBI/RHEL8 + Redis RPM
- **only if required by org:** keep Java 21 installed too

Keeping Java in the image makes it heavier for no Redis benefit.

Practical answer to your question

So yes, you should have these files:

- `Dockerfile`
- `redis.conf`
- `sentinel.conf`
- `entrypoint.sh`

And yes, I should have included `redis.conf` earlier.

If you want, next I'll give you a ↳ **complete OpenShift-ready folder structure** with:

- ↳ Dockerfile
- ↳ redis.conf
- ↳ sentinel.conf
- ↳ entrypoint.sh
- ↳ StatefulSet YAML
- ↳ Sentinel Deployment YAML
- ↳ Services
- ↳ PVC templates

yes please go ahead

Thought for 1m 59s

Done — I created the full starter package.

Download:
redis-sentinel-rhel8-openshift.tar.gz

It includes:

- `Dockerfile`
- `conf/redis.conf`
- `conf/sentinel.conf`
- `scripts/entrypoint.sh`
- raw OpenShift manifests under `k8s/`
- Helm chart under `helm/redis-sentinel-openshift/`

A couple of important notes:

- Update the image name in the YAML/Helm values to your internal registry image.
- Place your internal `redis-7.2*.rpm` in the build context before building.
- If your RPM installs Redis binaries in a non-standard path, adjust `scripts/entrypoint.sh`.
- If your OpenShift cluster requires a specific `storageClassName`, add it to the PVC specs.

Best next step is:

1. build the image from your RPM
2. push to internal registry
3. deploy with the raw YAML or Helm
4. run the included validation job
5. test failover by deleting `redis-0`

The package is a solid baseline, but I have not live-tested it on your OpenShift cluster, so expect a few environment-specific tweaks around storage, SCC, or image policy.

Clients should use Sentinel-aware Redis libraries or connect through an app-side discovery layer - what does it mean?

Great question — this is actually the **most important piece** to get right with Redis Sentinel.

Let's simplify it 👇

🧠 What does this mean?

> "Clients should use Sentinel-aware Redis libraries or connect through an app-side discovery layer"

👉 It means:

Your application **should NOT directly connect to a fixed Redis pod/service**.

Instead, it should:

- **ask Sentinel who the current primary is**
- then connect to that primary dynamically

🚨 Why this is needed

In Sentinel setup:

- Primary can **change anytime** (failover)
- Pod/service name you used earlier may **no longer be primary**

So if you hardcode:

```bash
redis://redis-0:6379
```

👉 After failover → ❌ broken app

✅ Correct way (2 approaches)

1️⃣ Sentinel-aware Redis client (BEST)

These clients **natively understand Sentinel**.

They:

1. connect to Sentinel
2. ask: *"who is master?"*
3. get IP/host of current primary
4. connect automatically
5. re-discover on failover

Example (Python)

Python Run

```python
from redis.sentinel import Sentinel

sentinel = Sentinel([
    ('sentinel-0', 26379),
    ('sentinel-1', 26379),
    ('sentinel-2', 26379)
])

redis_master = sentinel.master_for('mymaster')

redis_master.set('key', 'value')
print(redis_master.get('key'))
```

👉 No hardcoding of Redis primary

Example (Java - Jedis)

Java

```
Set<String> sentinels = new HashSet<>();
sentinels.add("sentinel-0:26379");
sentinels.add("sentinel-1:26379");

JedisSentinelPool pool = new JedisSentinelPool("mymaster", sentinels);

Jedis jedis = pool.getResource();
jedis.set("key", "value");
```

Example (Node.js)

JavaScript

```javascript
const Redis = require("ioredis");

const redis = new Redis({
  sentinels: [
    { host: "sentinel-0", port: 26379 },
    { host: "sentinel-1", port: 26379 }
  ],
  name: "mymaster"
});
```

👍 Benefits

- automatic failover handling
- no manual reconfiguration
- minimal operational overhead

2️⃣ App-side discovery layer

If your client library **does NOT support Sentinel**, then:

👉 You build a small layer in your app

How it works

1. App queries Sentinel manually:

```bash
SENTINEL get-master-addr-by-name mymaster
```

2. Gets:

```bash
["10.1.2.3", "6379"]
```

3. Connects Redis client to that IP

Pseudo-code

```python
master = get_master_from_sentinel()
redis = Redis(host=master.host, port=master.port)
```

❗ Problem

You must:

- re-query Sentinel on failure
- handle retries
- detect connection drops

👉 This is why using Sentinel-aware clients is preferred

🧩 In Kubernetes / OpenShift context

You will have:

- Sentinel service:

```bash
redis-sentinel:26379
```

- Redis pods:

```bash
redis-0
redis-1
redis-2
```

Client flow

```
App
  ↓
Sentinel Service
  ↓
Returns current primary (e.g., redis-1)
  ↓
App connects to redis-1
```

After failover

```
redis-0 dies
↓
Sentinel promotes redis-2
↓
Client reconnects automatically
```

❌ What NOT to do

```
App → redis service (round robin)
```

👉 This causes:

- writes hitting replicas ❌
- inconsistent data ❌
- errors ❌

💡 Important concept

Sentinel does NOT route traffic

It only:

- tells you where to go

👉 Your client must act on that information

🔑 Final takeaway

👉 Sentinel = “brain”
👉 Redis = “data”
👉 Client = “must listen to brain”

🧠 One-line summary

> A Sentinel-aware client dynamically discovers and connects to the current Redis primary using Sentinel, instead of hardcoding a Redis endpoint.

If you want next step, I can:

- ↳ show **how to wire this into your OpenShift service URLs**
- or ↳ give **production-ready connection configs (timeouts, retries, pooling)**
- or ↳ help integrate this into your **Playwright / Python automation stack**

my application which uses redis will be deployed in same cluster. How do you think it will connect to the redis. Can you give me examples. The applications can be java or python backend services

Thought for 7s ›

Yes. Since your app is in the **same OpenShift/Kubernetes cluster**, it will usually connect using **cluster DNS service names**, not pod IPs. With Redis Sentinel, the app should talk to the **Sentinel service** first, learn who the current Redis primary is, and then the client library opens the actual Redis connection to that primary. Sentinel is for discovery/failover, not for proxying application traffic. Home +2

A practical in-cluster setup looks like this:

- Redis pods: `redis-0`, `redis-1`, `redis-2`
- Redis headless service: `redis-headless`
- Sentinel pods: `sentinel-0`, `sentinel-1`, `sentinel-2`
- Sentinel service: `redis-sentinel`
- Redis master name in Sentinel: `mymaster`

So your application typically connects to:

- `redis-sentinel.<namespace>.svc.cluster.local:26379`

and gives the client library the Sentinel master name `mymaster`. The client then asks Sentinel for the current master and connects there. Spring Data Redis has explicit Sentinel configuration support for this model, and redis-py exposes Sentinel APIs such as `master_for(...)` and `slave_for(...)`. Home +2

What the connection flow looks like

For a write path, the flow is:

`App -> redis-sentinel service -> Sentinel returns current primary -> client connects to Redis primary`

If the primary fails and Sentinel promotes a replica, a Sentinel-aware client can discover the new primary and reconnect accordingly. That is exactly why you do not hardcode `redis-0:6379` in the app. Home +2

Python example

For Python backend services, the normal approach is `redis-py`, which Redis documents as the recommended Python client for most use cases. The `redis.sentinel.Sentinel` API supports discovering the master and getting a client bound to it. Redis +2

Python Run

```python
from redis.sentinel import Sentinel

# These are Kubernetes/OpenShift service DNS names.
# Usually a single Sentinel service is enough, but you can also list individual p
sentinel = Sentinel(
    [
        ("redis-sentinel.my-namespace.svc.cluster.local", 26379),
    ],
    socket_timeout=0.5,
    password=None,          # set if Sentinel itself is protected
    sentinel_kwargs={}      # e.g. {"password": "..."} if needed
)

# "mymaster" must match sentinel.conf -> sentinel monitor mymaster ...
redis_master = sentinel.master_for(
    "mymaster",
    socket_timeout=0.5,
    password=None,          # Redis password if enabled
    decode_responses=True
)

redis_master.set("xyz", "hello-from-python")
value = redis_master.get("xyz")
print(value)
```

If you want reads from replicas for non-critical read paths, redis-py also supports `slave_for(...)` via the Sentinel connection pool. That can be useful for read-heavy workloads, but for cache correctness and simplicity many teams keep both reads and writes on the primary. Redis Document... +1

In a FastAPI or Flask service, you would usually create this Sentinel client once during app startup and reuse the resulting pooled Redis client across requests, because redis-py uses connection pools under the hood. Redis Document... +1

Example with environment variables:

Python Run

```python
import os
from redis.sentinel import Sentinel

REDIS_MASTER_NAME = os.getenv("REDIS_MASTER_NAME", "mymaster")
REDIS_SENTINEL_SERVICE = os.getenv(
    "REDIS_SENTINEL_SERVICE",
    "redis-sentinel.my-namespace.svc.cluster.local"
```

```
)
REDIS_SENTINEL_PORT = int(os.getenv("REDIS_SENTINEL_PORT", "26379"))

sentinel = Sentinel(
    [(REDIS_SENTINEL_SERVICE, REDIS_SENTINEL_PORT)],
    socket_timeout=0.5
)

redis_client = sentinel.master_for(
    REDIS_MASTER_NAME,
    socket_timeout=0.5,
    decode_responses=True
)
```

You would then inject these env vars in your Deployment:

YAML

```yaml
env:
  - name: REDIS_MASTER_NAME
    value: "mymaster"
  - name: REDIS_SENTINEL_SERVICE
    value: "redis-sentinel.my-namespace.svc.cluster.local"
  - name: REDIS_SENTINEL_PORT
    value: "26379"
```

Java example with Jedis

For Java, Redis documents Jedis as a popular Java client, and Spring Data Redis has first-class Sentinel configuration support. If your service is plain Java or a lightweight framework, Jedis is straightforward. If it is Spring Boot, Spring Data Redis is usually the cleanest path. Redis +3

Plain Jedis example:

Java

```java
import redis.clients.jedis.Jedis;
import redis.clients.jedis.JedisSentinelPool;

import java.util.HashSet;
import java.util.Set;

public class RedisSentinelExample {
    public static void main(String[] args) {
        Set<String> sentinels = new HashSet<>();
```

```
        sentinels.add("redis-sentinel.my-namespace.svc.cluster.local:26379");

        try (JedisSentinelPool pool = new JedisSentinelPool("mymaster", sentinels
             Jedis jedis = pool.getResource()) {

            jedis.set("xyz", "hello-from-java");
            String value = jedis.get("xyz");
            System.out.println(value);
        }
    }
}
```

Jedis uses pooling, and current Redis docs note that its newer connection API provides pooling behavior automatically, so in an application you typically create the pool once and reuse it rather than re-creating it per request. Redis +1

Spring Boot example

If your Java backend is Spring Boot, configure Redis Sentinel in `application.yaml`. Spring Data Redis supports Sentinel mode directly through `RedisSentinelConfiguration` and the broader connection-modes configuration. Home +2

YAML

```yaml
spring:
  data:
    redis:
      sentinel:
        master: mymaster
        nodes:
          - redis-sentinel.my-namespace.svc.cluster.local:26379
      timeout: 2s
```

Then a normal `StringRedisTemplate` or `RedisTemplate` works:

Java

```java
@Service
public class CacheService {

    private final StringRedisTemplate redisTemplate;

    public CacheService(StringRedisTemplate redisTemplate) {
        this.redisTemplate = redisTemplate;
    }
```

```
    public void putValue(String key, String value) {
        redisTemplate.opsForValue().set(key, value);
    }

    public String getValue(String key) {
        return redisTemplate.opsForValue().get(key);
    }
}
```

That is usually the easiest enterprise-friendly setup because the failover handling is pushed into the Redis client stack rather than custom application code. Home +1

What service should the app use?

Use a **Sentinel service** for discovery, not a generic round-robin Redis service for writes. A normal Kubernetes service in front of all Redis pods would randomly send traffic to primary or replicas, which is wrong for writes. Sentinel-aware clients are meant to discover the current master and connect to the correct Redis node. Home +1

So the app should usually know only these values:

- Sentinel service DNS
- Sentinel port `26379`
- Sentinel master name, for example `mymaster`
- Redis auth secret, if enabled

It should **not** assume `redis-0` is always primary. Sentinel failover exists specifically because that may change. Home +1

OpenShift Deployment example

For a Python app:

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-python-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-python-app
  template:
```

```
    metadata:
      labels:
        app: my-python-app
    spec:
      containers:
        - name: app
          image: my-registry/my-python-app:latest
          env:
            - name: REDIS_MASTER_NAME
              value: "mymaster"
            - name: REDIS_SENTINEL_SERVICE
              value: "redis-sentinel.my-namespace.svc.cluster.local"
            - name: REDIS_SENTINEL_PORT
              value: "26379"
```

For a Spring Boot app:

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-java-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-java-app
  template:
    metadata:
      labels:
        app: my-java-app
    spec:
      containers:
        - name: app
          image: my-registry/my-java-app:latest
          env:
            - name: SPRING_DATA_REDIS_SENTINEL_MASTER
              value: "mymaster"
            - name: SPRING_DATA_REDIS_SENTINEL_NODES
              value: "redis-sentinel.my-namespace.svc.cluster.local:26379"
```

My recommendation for your setup

Since your apps are in the same cluster, I would keep it simple:

- expose **one Sentinel ClusterIP service**
- run **3 Sentinel pods**

- run **1 Redis primary + 2 replicas**
- configure the app with Sentinel master name + Sentinel service DNS
- use `redis-py` for Python and Spring Data Redis or Jedis for Java

That gives you clean service discovery, automatic failover handling, and no hardcoded primary pod name. Redis +2

The next useful step is for me to ↪ give you exact **application.properties / application.yaml**, Maven/Gradle dependencies, and Python `requirements.txt` for your stack.

Sources

What if I dont want to burden my client applications with this implementation of sentinel. They currently connect to single node redis. I want them to connect with minimal overhead

Thought for 7s

Yes — you can hide most of the Sentinel complexity from the applications.

The cleanest way is to keep **Redis Sentinel behind a stable internal endpoint** and let a **small proxy/discovery layer** deal with failover, because Redis Sentinel itself is a discovery system and official Redis docs say clients need explicit Sentinel support when talking to Sentinel directly. Redis +1

Best fit for your requirement

Since your apps already expect **one Redis endpoint**, I'd recommend this pattern:

App → stable service/proxy → current Redis primary

instead of:

App → Sentinel directly

That gives you:

- minimal or no app code changes

- one hostname/port for apps
- Sentinel still handles failover in the backend
- proxy layer switches traffic to the new primary after failover

Two practical options

Option 1: HAProxy in front of Redis primary

This is usually the most practical.

HAProxy can do TCP health checks, and it supports Redis-aware checks such as sending `INFO replication` and verifying `role:master`, so it can keep only the current master as active. HAProxy documents both generic TCP checks and Redis-specific `tcp-check` patterns for master detection. HAProxy Techno... +1

So your architecture becomes:

Java/Python app
→ `redis-proxy.my-namespace.svc:6379`
→ HAProxy
→ current Redis primary

When failover happens:

- Sentinel promotes a replica
- HAProxy health checks detect the new `role:master`
- apps continue using the same endpoint

This is the **lowest-overhead app-side model**.

Option 2: App-side discovery wrapper

If you do not want every service to understand Sentinel, you can build one small internal library or sidecar that queries Sentinel and returns the active primary. This still uses Sentinel semantics, but only one shared component owns the complexity. Redis docs describe Sentinel as a configuration provider for service discovery, where clients ask Sentinel for the current master address. Redis +1

This is better than changing every service individually, but it is still more work than using a proxy.

What I would recommend for your org

For your case, I would choose:

Redis primary + replicas + Sentinel + HAProxy service

Why:

- your apps can keep using **a single Redis host**
- almost no code changes
- works for both Java and Python services
- easier migration from single-node Redis
- Sentinel still gives HA and failover in the backend

What apps would use

Instead of this old style:

```
redis-single:6379
```

they would use:

```
redis-proxy.my-namespace.svc.cluster.local:6379
```

That stays constant.

Java example

If your app already uses a normal Redis client and talks to a single host, it can keep doing that.

With Jedis:

```Java
import redis.clients.jedis.JedisPooled;

public class RedisClientExample {
    public static void main(String[] args) {
        JedisPooled jedis = new JedisPooled("redis-proxy.my-namespace.svc.cluster

        jedis.set("xyz", "hello");
        String value = jedis.get("xyz");
        System.out.println(value);
    }
}
```

No Sentinel logic in the app.

With Spring Boot, it can stay similarly simple:

```yaml
spring:
  data:
    redis:
      host: redis-proxy.my-namespace.svc.cluster.local
      port: 6379
      timeout: 2s
```

Python example

Also simple:

```python
import redis

r = redis.Redis(
    host="redis-proxy.my-namespace.svc.cluster.local",
    port=6379,
    decode_responses=True,
    socket_timeout=2
)

r.set("xyz", "hello")
print(r.get("xyz"))
```

Again, no Sentinel code in the application.

Trade-offs

This simpler model is great, but you should know the trade-offs:

- during failover, there may be a short reconnect window while Sentinel elects a new primary and the proxy detects it
- writes still go to only one primary at a time
- replication is still asynchronous, so a small lag window can exist between primary and replicas in Redis replication. Redis documents replication as leader-follower and notes replicas aim to become exact copies of the master, reconnecting automatically after link breaks. Redis

My simple recommendation

If you want **minimal client overhead**, do this:

Preferred architecture

- 1 Redis primary
- 2 Redis replicas
- 3 Sentinel pods
- 1 HAProxy service in front of Redis
- apps connect only to HAProxy

Avoid

- making every app Sentinel-aware unless you want direct failover intelligence in each service
- putting a normal round-robin Kubernetes Service in front of all Redis pods for writes, because traffic could land on replicas instead of the primary

If you want, I'll ↳ generate the **OpenShift HAProxy layer** for your existing Redis Sentinel package too, including:

- ↳ Dockerfile
- `haproxy.cfg`
- ↳ Service
- ↳ Deployment
- ↳ health checks for current master detection

Sources

Okey what if I use redis cluster that you recommended previously? I think sharding gives me some confidence

Thought for 6s ›

You *can* use Redis Cluster, but it changes the trade-off quite a bit.

For your goals, the big question is not "is sharding good?" — it is:

Do you want scale more than client simplicity?

If you use Redis Cluster

You get:

- **horizontal sharding** of data across masters
- replicas per shard for failover
- better scale for memory and throughput

But you also accept:

- **keys are split across shards**, not copied to every pod
- clients generally need to be **cluster-aware**
- some commands and key patterns become trickier in clustered mode

Redis Cluster distributes the keyspace across **16384 hash slots**, and each slot is owned by one shard at a time. A shard can have one master and replicas. Also, Redis documents that to connect to Redis Cluster, you need a **cluster-aware Redis client**. Redis +2

What happens to your "single endpoint" expectation

This is the key downside for your current app model.

In Redis OSS Cluster, **nodes do not proxy requests for each other**. If a client connects to the wrong node for key `xyz`, Redis returns a `MOVED` or sometimes `ASK` redirection, and the client must retry against the correct node. Redis states this explicitly in the cluster spec. Redis

So compared with Sentinel:

- **Sentinel** can be hidden behind a proxy more easily
- **Cluster** usually pushes more intelligence into the client layer

That means if your applications currently expect:

- one host
- one port
- normal Redis commands
- no failover logic
- no shard awareness

then **Redis Cluster is not the lowest-overhead migration path**. Redis +1

Will sharding give you "confidence"?

Yes, but only in a specific sense.

It gives confidence for:

- growing dataset size
- distributing load
- avoiding one huge Redis primary becoming a bottleneck

It does **not** give confidence in the "same data on every node" sense. In cluster mode, `xyz` lives on the shard that owns its slot, with replicas for HA. It is not available locally on every pod. Redis +1

My recommendation for your case

Based on everything you said so far, I'd frame it like this:

Choose Redis Cluster if:

- you expect cache size or throughput to outgrow one Redis primary
- your Java/Python clients can use cluster-aware libraries
- you are okay with sharded data
- you can tolerate a bit more operational and client complexity

Choose Redis Sentinel if:

- your apps should keep using something close to a **single Redis endpoint**
- you want minimal application changes
- your dataset fits on one primary with replicas
- your priority is HA and simplicity, not horizontal scale

What I'd do in your environment

I would use this decision rule:

- If your cache is likely to stay within the limits of **one strong Redis node**, use **Sentinel**
- If you genuinely expect **large growth in memory / QPS** and want to scale out early, use **Cluster**

For most organizations migrating from a single Redis node with minimal app change, **Sentinel is the safer first step**. Then move to Cluster only when scale becomes a real bottleneck.

Practical guidance for Java and Python

If you choose Cluster, your apps should use:

- **Jedis Cluster** or **Lettuce Cluster** in Java
- a **cluster-capable redis-py setup** or another cluster-aware Python client, depending on the client stack you standardize on

Because Redis Cluster requires clients to follow redirections and understand slot ownership. Redis's own docs make that requirement clear. Redis +1

My honest bottom line

For **your stated requirement** of minimal client burden, I would still prefer:

Sentinel + stable proxy endpoint

over

Redis OSS Cluster

But if you want to optimize for future scale and are comfortable standardizing cluster-aware libraries in Java and Python, then Redis Cluster is a valid choice.

The next useful step is for me to give you a **side-by-side architecture recommendation with exact Java/Python client examples for both Sentinel and Cluster**.

Sources

A curiosity question, let us say we I have multiple clients who will be connection to my redis . How would I handle them?

Thought for 6s

You handle them the same way you handle many clients for any shared backend: **don't give every request its own brand-new Redis connection**, and **put a stable access pattern in front of Redis**.

For your case, with multiple Java/Python services connecting from the same cluster, I'd think about it in 3 layers:

1. How clients connect

Each application instance should use a **Redis client pool** or multiplexed connection model, not open/close a socket for every call. Redis documents **connection pooling and multiplexing** as the two standard approaches, and specifically notes that `redis-py`,

`jedis`, and `go-redis` support pooling, while Lettuce supports both pooling and multiplexing. Redis

That means:

- **Python** service: create one Redis client / pool per process
- **Java** service: create one pool or shared connection factory per app instance
- **Do not** create a new Redis client inside every request handler

So if you have:

- 10 pods of service A
- 5 pods of service B
- 3 pods of service C

then Redis sees a manageable number of long-lived pooled connections rather than thousands of short-lived connects/disconnects.

---

2. How Redis handles many connected clients

Redis is designed to handle many client connections, but there is still a server-side limit. Redis has a `maxclients` setting that controls the maximum number of simultaneous client connections, and its client-handling docs cover timeouts, buffers, and related network behavior. You can inspect current clients with `CLIENT LIST`. Redis +1

So operationally, you should plan for:

- total app pods × average pool size
- a few admin/monitoring connections
- some headroom for reconnect storms during failover or pod restarts

Example:

- 20 app pods
- each keeps pool size 20

That is roughly **400 active/possible client connections** already, before admin tools.

That is not a problem by itself, but it should be intentional.

---

3. What architecture to use for many clients

This depends on whether you choose **Sentinel** or **Cluster**.

If you use Sentinel

Best when:

- you want apps to feel like they connect to **one logical Redis**
- dataset fits on one primary
- you want minimal change in clients

Pattern:

- many clients connect through a **stable endpoint**
- either:
  - Sentinel-aware libraries, or
  - a proxy/discovery layer in front of Redis

Redis documents Sentinel as a **service discovery and failover** mechanism, and says clients need explicit Sentinel support if they talk to Sentinel directly. Redis +1

For many clients, I would usually recommend:

Apps → one stable Redis endpoint/proxy → current primary

That way all your services still think they are using one Redis endpoint.

If you use Cluster

Best when:

- you expect many clients and high throughput
- you want to scale horizontally
- you are okay with sharding

Redis Cluster spreads keys over **16384 hash slots**, and clients are expected to be **cluster-aware** because they may get redirected with `MOVED` or `ASK` to the right node. Redis +2

For many clients, Cluster can actually be a good fit because:

- traffic is distributed across shards
- one node does not handle the whole dataset

But the client side is more complex.

Simple recommendation for “many clients”

If your concern is:

“I will have many backend services connecting”

then do this:

With Sentinel

- use **1 primary + 2 replicas + 3 sentinels**
- put a **stable internal service/proxy** in front
- every app connects to that one endpoint
- each app uses a connection pool

This is the easiest migration from single-node Redis.

With Cluster

- use a **cluster-aware client**
- bootstrap with all cluster nodes or a cluster service
- let the client manage slot routing
- still use pooling in each app

This is better if you expect scale/load growth.

Practical concerns when many clients connect

These are the real things to watch:

Connection storms

If Redis restarts or failover happens, many app pods may reconnect together. Pooling helps, but you should still set sane reconnect/backoff settings.

Pool sizing

Don’t set huge pool sizes “just in case.”
A pool of 100 per pod across many services can become wasteful fast.

Timeouts

Use reasonable:

- connect timeout
- read timeout
- retry/backoff

Monitoring

Track:

- connected clients
- blocked clients
- memory
- replication lag
- command latency

Redis's client-handling docs and `CLIENT LIST` / `INFO` are the starting points for this. Redis +2

Security

If many apps connect, use Redis **ACLs** rather than one shared password for everything. Redis ACLs let you restrict commands and keys by user. Redis

---

What I would do in your org

Because you want **minimal overhead for clients**, I would choose:

Option A — simplest

Redis Sentinel + HAProxy + connection pools in apps

- apps keep using one host/port
- HA/failover is hidden
- many clients are handled through normal client pools

Option B — scale-first

Redis Cluster + cluster-aware Java/Python clients + modest pooling

- better long-term scale
- more client awareness required

Very short answer

If multiple clients connect to Redis:

- Redis can handle many clients, but size your **connection pools** properly and monitor `maxclients`. Redis +1

- With **Sentinel**, hide failover behind a stable endpoint if you want minimal app changes. Redis +1
- With **Cluster**, use **cluster-aware clients** because requests are routed by hash slot, not by one shared primary. Redis +1

For your situation, I'd still lean **Sentinel + stable proxy endpoint** unless you already know you need horizontal scale.

Sources

I can go with
With Sentinel
use 1 primary + 2 replicas + 3 sentinels
put a stable internal service/proxy in front
every app connects to that one endpoint
each app uses a connection pool

This is the easiest migration from single-node Redis.

Thought for 7s

Yes — that is the **right choice for your migration path**.

For your requirements, I'd recommend this target architecture:

Apps → stable internal proxy service → current Redis primary
with **1 primary + 2 replicas + 3 Sentinels** behind it.

That gives you automatic failover from Sentinel, while keeping the application view close to your current "single-node Redis" model. Redis Sentinel is designed for automatic failover and service discovery, and clients are normally expected to be Sentinel-aware if they talk to Sentinel directly. Using a stable proxy endpoint avoids pushing that logic into every app. Redis +3

Why this is a good fit

- **Minimal app change**: apps keep using one host/port.
- **HA**: if the primary fails, Sentinel promotes a replica.
- **Operationally simpler than Redis Cluster** for your use case.

- **Connection pooling** in the app keeps Redis connection handling efficient and avoids opening a fresh socket per request. Redis recommends pooling/multiplexing patterns, and Redis docs explicitly recommend connection pooling in general. Redis +3

What I would standardize

Use these components:

- **Redis StatefulSet**: 3 pods
  - `redis-0` = primary
  - `redis-1`, `redis-2` = replicas
- **Sentinel**: 3 pods
- **HAProxy**: 2 pods behind one ClusterIP service
- **One stable service name for apps**, for example: `redis.my-namespace.svc.cluster.local:6379`

HAProxy can health-check Redis and detect which backend is the current master, so it can keep routing writes to the right node after failover. HAProxy Techno... +1

What your apps do

Your Java and Python services can continue using a normal Redis client against one endpoint:

- Java/Spring: `spring.data.redis.host=redis.my-namespace.svc.cluster.local`
- Python: `redis.Redis(host="redis.my-namespace.svc.cluster.local", port=6379, ...)`

Each app should still use a **connection pool**, not create a new Redis connection per request. Redis and Jedis both document pooled connection handling as the recommended model. Redis +2

Important caveat

This is still **single-writer Redis**:

- all writes go to the current primary
- replicas follow asynchronously
- during failover there can be a brief reconnect window

So this gives you **high availability with minimal app overhead**, not zero-disruption strong consistency. Redis Sentinel's failover and Redis client handling docs both reflect that model. Redis +1

My recommendation

Go ahead with:

- **1 primary**
- **2 replicas**
- **3 sentinels**
- **HAProxy in front**
- **one stable service for apps**
- **connection pooling in each app**

That is the cleanest first step from single-node Redis.

The next most useful thing is for me to generate the **HAProxy layer and updated OpenShift manifests/Helm chart** on top of the Sentinel package so your applications can connect to one stable endpoint.

Sources

Generate the HAProxy layer and updated OpenShift manifests/Helm chart on top of the Sentinel package

Thought for 1m 58s

Done — I created the updated package with the **HAProxy layer** and **OpenShift manifests + Helm chart** on top of the Sentinel setup.

Download:
redis-sentinel-rhel8-openshift-haproxy.tar.gz

What's included:

- `Dockerfile`
- `conf/redis.conf`
- `conf/sentinel.conf`
- `conf/haproxy.cfg`
- `scripts/entrypoint.sh`
- raw manifests in `k8s/`
- Helm chart in `helm/redis-sentinel-openshift-proxy/`

What changed:

- added **HAProxy** into the image
- added **stable internal service**: `redis-proxy`
- HAProxy health-checks Redis nodes and routes to the **current primary**
- apps can now keep using a **single Redis host/port**
- included HAProxy stats service on port `16379`

Recommended app endpoint:

- `redis-proxy.<namespace>.svc.cluster.local:6379`

Two practical notes:

- the HAProxy config currently assumes **3 Redis pods**: `redis-0`, `redis-1`, `redis-2`
- update the image reference and storage class before deploying

Best validation path:

1. build image from your internal Redis 7.2 RPM
2. push to internal registry
3. deploy manifests or Helm chart
4. run the validation job
5. delete `redis-0` and verify `redis-proxy` still serves traffic after failover

I have not live-tested it on your OpenShift cluster, so expect small environment-specific tweaks around image pull policy, SCC behavior, or storage class names.